

Prompts Para IA Cree los Diagramas

Diagrama de casos de Uso

“Necesito generar un diagrama de casos de uso UML para un proyecto de Ingeniería de Software llamado BowlingPro.

BowlingPro es una aplicación móvil orientada a jugadores en formación de bowling y entrenadores. El sistema permite apoyar el aprendizaje, entrenamiento, seguimiento del progreso, comunicación y administración del contenido y las cuentas.

Quiero que generes el diagrama de casos de uso completo, claro, ordenado y visualmente limpio.

ACTORES Y CASOS DE USO

1. Jugador

- Registrarse
- Iniciar sesión
- Consultar perfil
- Aprendizaje con lecciones
- Evaluación
- Registrar partida
- Estadísticas
- Entrenamiento
- Retos
- Información
- Comunicación
- Notificaciones

2. Entrenador

- Registrarse
- Iniciar sesión
- Gestionar perfil
- Consultar jugadores
- Gestionar planes
- Consultar progreso / estadísticas / historial
- Retroalimentación
- Observaciones

- Comunicación

3. Administrador

- Iniciar sesión
- Gestionar cuentas
- Gestionar contenido

REGLAS DEL DIAGRAMA

1. Representa a BowlingPro como el límite del sistema, utilizando un contenedor claramente identificado como "BowlingPro".
2. Los tres actores deben aparecer fuera del límite del sistema:
 - Jugador
 - Entrenador
 - Administrador
3. Cada actor debe estar conectado únicamente con los casos de uso que puede realizar.
4. Utiliza asociaciones normales entre actores y casos de uso cuando no exista una relación especial.
5. No agregues casos de uso nuevos que no estén en la lista proporcionada.
6. No inventes relaciones <<include>> o <<extend>>. Si no existe suficiente información para justificar una relación, utiliza únicamente una asociación normal.
7. No conectes todos los casos de uso entre sí. Las relaciones deben representar únicamente interacciones justificables dentro del sistema.
8. "Iniciar sesión" debe mantenerse como un caso de uso independiente para cada actor que lo utiliza. No es necesario conectar "Iniciar sesión" mediante <<include>> con todas las demás funcionalidades.
9. Mantén los nombres de los casos de uso exactamente como fueron proporcionados, corrigiendo únicamente errores ortográficos evidentes.
10. Organiza visualmente el diagrama para minimizar el cruce de líneas:
 - Jugador en el lado izquierdo.
 - BowlingPro en el centro.
 - Entrenador en la parte derecha.
 - Administrador en una zona inferior o superior derecha.
11. El resultado debe parecer un diagrama UML de casos de uso académico, sencillo y profesional, adecuado para una entrega universitaria.

12. Si Mermaid no permite representar perfectamente la notación UML tradicional, utiliza la aproximación más clara posible sin alterar los casos de uso ni las relaciones.

RESULTADO ESPERADO

Entrega únicamente:

1. El código Mermaid completo.
2. Una explicación muy breve de las asociaciones utilizadas y de cualquier relación <<include>> o <<extend>>, únicamente si decides que alguna está realmente justificada.

No agregues funcionalidades, actores ni relaciones que no hayan sido especificados anteriormente.”

Diagrama De Clases

“Crea un DIAGRAMA UML DE CLASES para un sistema llamado "BowlingPro".

BowlingPro es una aplicación móvil orientada a jugadores de bowling en formación.

La aplicación permite aprender técnicas de bowling, realizar evaluaciones, registrar partidas,

calcular automáticamente el puntaje oficial, consultar estadísticas, realizar planes de

entrenamiento, recibir retroalimentación de entrenadores, participar en retos y obtener logros.

IMPORTANTE:

- Utiliza exactamente las 18 clases indicadas a continuación.
- No agregues clases adicionales.
- No elimines ninguna clase.
- Utiliza notación UML estándar.
- Mostrar atributos y métodos principales dentro de cada clase.

- Mostrar relaciones y multiplicidades.
- Organizar visualmente las clases por módulos para que el diagrama sea fácil de leer.
- No incluir código de implementación ni detalles de base de datos.
- Utilizar nombres de clases, atributos y métodos en español.
- La clase Usuario debe ser una clase general/abstracta de la que hereden Jugador, Entrenador y Administrador.

=====

1. MÓDULO DE USUARIOS

=====

CLASE: Usuario

Atributos:

- idUsuario
- nombre
- correo
- contrasena
- rol

Métodos:

- registrarse()
- iniciarSesion()
- cerrarSesion()

CLASE: Jugador

Hereda de Usuario.

Atributos:

- nivelActual

Métodos:

- consultarPerfil()
- consultarProgreso()
- consultarEstadisticas()
- registrarPartida()
- consultarHistorial()

CLASE: Entrenador

Hereda de Usuario.

Métodos:

- consultarJugadoresAsignados()
- asignarPlan()
- ajustarPlan()
- consultarProgresoJugador()
- registrarRetroalimentacion()
- registrarObservacion()

CLASE: Administrador

Hereda de Usuario.

Métodos:

- gestionarCuentas()

- crearContenido()
- editarContenido()
- eliminarContenido()

=====

2. MÓDULO DE APRENDIZAJE

=====

CLASE: Nivel

Atributos:

- idNivel
- nombre
- descripcion
- orden

Métodos:

- desbloquear()

CLASE: Leccion

Atributos:

- idLeccion
- titulo
- descripcion
- tipoContenido
- orden

Métodos:

- consultarContenido()

CLASE: Evaluacion

Atributos:

- idEvaluacion

- pregunta

- respuestaCorrecta

- notaMinima

Métodos:

- calificar()

- validarCalificacion()

CLASE: ProgresoAprendizaje

Atributos:

- idProgreso

- porcentaje

- estado

- fechaActualizacion

Métodos:

- actualizarProgreso()

- calcularPorcentaje()

=====

3. MÓDULO DE PARTIDAS Y ESTADÍSTICAS

=====

CLASE: Partida

Atributos:

- idPartida
- fecha
- tipo
- puntajeTotal

Métodos:

- registrarLanzamiento()
- calcularPuntaje()
- finalizarPartida()

CLASE: Lanzamiento

Atributos:

- idLanzamiento
- numero
- pinosDerribados
- esStrike
- esSpare

Métodos:

- registrar()
- determinarStrike()
- determinarSpare()

CLASE: Estadística

Atributos:

- idEstadistica
- promedio
- porcentajeStrike
- porcentajeSpare
- pinosPorPartida

Métodos:

- calcularPromedio()
- calcularPorcentajeStrike()
- calcularPorcentajeSpare()
- calcularPinosPorPartida()

=====

4. MÓDULO DE ENTRENAMIENTO

=====

CLASE: PlanEntrenamiento

Atributos:

- idPlan
- nombre
- descripcion
- fechaInicio
- fechaFin

Métodos:

- asignar()

- ajustar()

CLASE: Ejercicio

Atributos:

- idEjercicio

- nombre

- descripcion

- tipo

Métodos:

- consultar()

CLASE: ProgresoEjercicio

Atributos:

- idProgreso

- completado

- fechaCompletado

Métodos:

- marcarCompletado()

=====

5. MÓDULO DE RETOS Y LOGROS

=====

CLASE: Reto

Atributos:

- idReto
- nombre
- descripcion
- condicion

Métodos:

- validar()

CLASE: Logro

Atributos:

- idLogro
- nombre
- descripcion
- insignia

Métodos:

- desbloquear()

=====

6. MÓDULO DE COMUNICACIÓN

=====

CLASE: Retroalimentacion

Atributos:

- idRetroalimentacion

- comentario
- fecha
- tipo

Métodos:

- registrar()
- consultar()

CLASE: Notificacion

Atributos:

- idNotificacion
- tipo
- mensaje
- fecha
- leida

Métodos:

- enviar()
- marcarComoLeida()

=====

RELACIONES Y MULTIPLICIDADES

=====

HERENCIA:

Usuario <|-- Jugador

Usuario <|-- Entrenador

Usuario <|-- Administrador

APRENDIZAJE:

Nivel "1" ----- "1..*" Leccion

Leccion "1" ----- "0..*" Evaluacion

Jugador "1" ----- "1" ProgresoAprendizaje

ProgresoAprendizaje "1" ----- "0..*" Leccion

ProgresoAprendizaje "1" ----- "1" Nivel

PARTIDAS:

Jugador "1" ----- "0..*" Partida

Partida "1" ----- "1..*" Lanzamiento

Jugador "1" ----- "1" Estadistica

ENTRENAMIENTO:

Entrenador "1" ----- "0..*" PlanEntrenamiento

Jugador "1" ----- "0..*" PlanEntrenamiento

PlanEntrenamiento "1" ----- "1..*" Ejercicio

Jugador "1" ----- "0..*" ProgresoEjercicio

Ejercicio "1" ----- "0..*" ProgresoEjercicio

RETOS Y LOGROS:

Jugador "0..*" ----- "0..*" Logro

El Reto se valida utilizando el historial de Partida del jugador.

Representar esta relación de forma conceptual si la herramienta lo considera necesario, pero NO crear una clase adicional.

COMUNICACIÓN:

Entrenador "1" ----- "0..*" Retroalimentacion

Jugador "1" ----- "0..*" Retroalimentacion

Usuario "1" ----- "0..*" Notificacion

=====

REGLAS DE NEGOCIO QUE DEBE REFLEJAR EL MODELO

=====

1. El jugador avanza de nivel únicamente cuando completa las lecciones y evaluaciones correspondientes y obtiene la calificación mínima requerida.
2. El puntaje de una partida debe calcularse utilizando las reglas oficiales de bowling, incluyendo strikes y spares.
3. No se puede acceder a lecciones de niveles superiores sin completar previamente los niveles anteriores.
4. Los retos se validan utilizando información registrada en el historial de partidas del jugador.
5. Un entrenador solamente puede consultar y proporcionar retroalimentación a los jugadores que tiene asignados.

6. Solamente el administrador puede crear, editar o eliminar contenido de aprendizaje.

REQUISITOS DE PRESENTACIÓN

- Utilizar UML estándar.
- Mostrar claramente la herencia.
- Mostrar las multiplicidades en las asociaciones.
- Separar visualmente los módulos:

Usuarios

Aprendizaje

Partidas y Estadísticas

Entrenamiento

Retos y Logros

Comunicación

- Mantener el diagrama organizado y legible.
- Evitar cruces innecesarios de líneas.
- Si es necesario, distribuir los módulos en diferentes zonas del diagrama.
- No crear clases adicionales.
- No convertir cada regla de negocio en una clase.
- No crear clases para "Puntuacion", "Clasificacion", "Informacion", "Glosario", "Regla", "JugadorLogro" ni otras clases adicionales.

- El resultado debe representar un **DIAGRAMA DE CLASES CONCEPTUAL** para un

proyecto académico de Ingeniería de Software, no un modelo de base de datos.

Título del diagrama: "Diagrama de Clases - BowlingPro"

Diagrama de Actividad

Crear una imagen única que contenga **tres diagramas de actividades UML de BowlingPro**, uno para cada tipo de usuario: **Jugador en formación, Entrenador y Administrador**.

Los tres diagramas deben estar dentro de la misma imagen, claramente separados mediante títulos, pero manteniendo exactamente el mismo estilo visual, tamaño de elementos, tipografía y simbología UML.

Contexto del sistema

BowlingPro es una aplicación móvil para jugadores en formación de bowling. Permite aprender técnicas mediante lecciones, registrar partidas, calcular puntuaciones, consultar estadísticas y progreso, realizar ejercicios, recibir entrenamiento personalizado, participar en retos y obtener logros. Los entrenadores pueden gestionar planes de entrenamiento y hacer seguimiento de sus jugadores. El administrador gestiona cuentas y contenido educativo.

Los diagramas deben ser coherentes con el siguiente modelo de clases:

- Usuario
- Jugador
- Entrenador
- Administrador
- Nivel
- Lección
- Evaluación
- ProgresoAprendizaje
- Partida
- Lanzamiento
- Estadística
- PlanEntrenamiento
- Ejercicio
- ProgresoEjercicio

- Reto
- Logro
- Retroalimentacion
- Notificacion

Las relaciones principales del modelo son:

- Usuario es la clase general de la cual heredan Jugador, Entrenador y Administrador.
- Nivel contiene Lecciones.
- Lección contiene Evaluaciones.
- Jugador tiene ProgresoAprendizaje.
- Jugador registra Partidas.
- Partida contiene Lanzamientos y genera Estadísticas.
- Entrenador gestiona PlanEntrenamiento.
- PlanEntrenamiento contiene Ejercicios.
- Jugador registra ProgresoEjercicio.
- Jugador puede obtener Logros mediante Retos.
- Entrenador registra Retroalimentacion para sus jugadores.
- Los usuarios reciben Notificacion.

DIAGRAMA 1 — ACTIVIDADES DEL JUGADOR EN FORMACIÓN

Título: **Diagrama de actividades — Jugador en formación**

Crear un flujo UML que comience con:

Inicio → Iniciar sesión → Validar credenciales

Agregar una decisión:

¿Credenciales válidas?

- Si NO → Mostrar mensaje de error → Volver a ingresar credenciales.
- Si SÍ → Acceder a BowlingPro → Mostrar pantalla principal.

Desde la pantalla principal, representar una decisión o bifurcación de navegación donde el jugador pueda seleccionar diferentes funcionalidades.

Rama 1: Aprendizaje

Consultar niveles → Seleccionar nivel → Consultar lecciones → Seleccionar lección → Consultar contenido → Realizar evaluación → Actualizar ProgresoAprendizaje

Agregar una decisión:

¿Cumple requisitos para avanzar?

- Sí → Desbloquear siguiente nivel.
- No → Mantener nivel actual.

Rama 2: Partidas y estadísticas

Registrar partida → Registrar Lanzamientos → Calcular puntuación oficial → Guardar Partida → Actualizar Estadística → Consultar estadísticas/historial

El cálculo debe contemplar la lógica de bowling para strikes, spares, lanzamientos y los 10 frames, pero mantenerlo visualmente como una actividad del sistema sin entrar en fórmulas matemáticas.

Rama 3: Entrenamiento

Consultar PlanEntrenamiento → Consultar Ejercicios → Seleccionar ejercicio → Realizar ejercicio → Registrar ProgresoEjercicio → Actualizar progreso

Rama 4: Retos y logros

Consultar Retos → Seleccionar reto → Verificar condición con historial de Partidas/Estadísticas

Agregar decisión:

¿Cumple condición del reto?

- Sí → Validar reto → Desbloquear Logro → Generar Notificacion.
- No → Mostrar reto pendiente.

Rama 5: Seguimiento y comunicación

Permitir:

Consultar Retroalimentacion → Comunicarse con Entrenador → Recibir Notificacion

También incluir:

Consultar perfil

El diagrama debe mostrar que estas funcionalidades son opciones de navegación desde la pantalla principal y **no pasos obligatoriamente consecutivos**.

Todas las actividades deben terminar en un nodo de **Fin**.

DIAGRAMA 2 — ACTIVIDADES DEL ENTRENADOR

Título: **Diagrama de actividades — Entrenador**

Comenzar con:

Inicio → Iniciar sesión → Validar credenciales

Decisión:

¿Credenciales válidas?

- No → Mostrar error → Volver al inicio de sesión.
- Sí → Acceder al panel del entrenador.

Desde el panel representar las diferentes funcionalidades disponibles.

Rama 1: Gestión y seguimiento de jugadores

Consultar jugadores asignados → Seleccionar jugador → Consultar progreso → Consultar estadísticas → Consultar historial de partidas

Respetar la regla de negocio de que el entrenador solamente puede consultar información de **jugadores que tiene asignados**.

Rama 2: Plan de entrenamiento

Consultar PlanEntrenamiento → Crear/modificar plan → Seleccionar Ejercicios → Asignar plan al jugador → Guardar plan

Rama 3: Seguimiento del entrenamiento

Consultar progreso de ejercicios → Registrar observación → Registrar Retroalimentación → Enviar retroalimentación al jugador → Generar Notificación

Rama 4: Perfil

Consultar perfil → Actualizar información → Guardar cambios

Rama 5: Comunicación

Seleccionar jugador → Comunicarse con jugador → Enviar mensaje/retroalimentación → Generar Notificación

Las diferentes ramas deben representar funcionalidades seleccionables desde el panel del entrenador y no una secuencia obligatoria.

Todas las operaciones deben terminar regresando al panel o al nodo de **Fin**.

DIAGRAMA 3 — ACTIVIDADES DEL ADMINISTRADOR

Título: **Diagrama de actividades — Administrador**

Comenzar con:

Inicio → Iniciar sesión → Validar credenciales

Decisión:

¿Credenciales válidas?

- No → Mostrar error → Volver al inicio de sesión.
- Sí → Acceder al panel de administración.

Desde el panel representar dos grandes ramas.

Rama 1: Gestionar cuentas

Gestionar cuentas → Consultar cuentas → Seleccionar cuenta

Agregar una decisión:

¿Qué operación realizar?

- Crear cuenta → Registrar datos → Validar información → Guardar cuenta.
- Modificar cuenta → Actualizar datos → Validar información → Guardar cambios.
- Eliminar cuenta → Confirmar eliminación → Eliminar cuenta.

Rama 2: Gestionar contenido educativo

Gestionar contenido educativo → Consultar contenido → Seleccionar contenido

Agregar una decisión:

¿Qué operación realizar?

- Crear contenido → Registrar información → Guardar contenido.
- Editar contenido → Modificar información → Guardar cambios.
- Eliminar contenido → Confirmar eliminación → Eliminar contenido.

El contenido educativo debe estar relacionado conceptualmente con las clases **Nivel, Lección y Evaluación**.

Después de cada operación, regresar al panel de administración o permitir seleccionar otra funcionalidad.

Finalizar con un nodo de **Fin**.

Reglas visuales para los tres diagramas

Utilizar simbología UML estándar:

- Círculo negro para **Inicio**.
- Rectángulos con esquinas redondeadas para **actividades**.
- Rombo para **decisiones**.
- Flechas para indicar el flujo.
- Círculo de finalización UML para **Fin**.
- Etiquetas **[Sí]** y **[No]** en las decisiones.
- Utilizar bifurcaciones solamente cuando realmente representen opciones o actividades paralelas.
- Mantener los flujos ordenados y fáciles de seguir.
- Evitar cruces innecesarios entre flechas.

- No convertir cada clase del diagrama de clases en una actividad; utilizar las clases como respaldo conceptual de las funcionalidades.
- No agregar funcionalidades que no estén relacionadas con los requisitos de BowlingPro.
- Mantener nombres exactamente como aparecen en el modelo: **Jugador, Entrenador, Administrador, Nivel, Lección, Evaluación, ProgresoAprendizaje, Partida, Lanzamiento, Estadística, PlanEntrenamiento, Ejercicio, ProgresoEjercicio, Reto, Logro, Retroalimentacion y Notificacion.**
- Usar español.
- Mantener los tres diagramas visualmente consistentes.
- El resultado debe parecer un trabajo académico de Ingeniería de Software/UML, limpio, profesional y legible.

Distribución de la imagen

Crear una sola imagen horizontal o vertical de alta resolución con los tres diagramas claramente separados:

DIAGRAMA DE ACTIVIDADES — JUGADOR EN FORMACIÓN

[diagrama]

DIAGRAMA DE ACTIVIDADES — ENTRENADOR

[diagrama]

DIAGRAMA DE ACTIVIDADES — ADMINISTRADOR

[diagrama]

No mezclar los flujos de los tres roles. Cada diagrama debe ser independiente y completo.